In [2]:
```
import pandas as pd
import numpy as np
from sklearn.datasets import fetch_openml
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
```

In [3]:
```
mnist = fetch_openml("mnist_784",version=1, as_frame=False)
X = mnist.data.astype("float32")
y = mnist.target.astype(int)
print("Dataset shape:", X.shape)
X = X / 255.0
X = X[:3000]
y = y[:3000]
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 0.2, random_st
```

```
Dataset shape: (70000, 784)
```

In [4]:
```
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

In [5]:
```
#Linear Model
linear_svm = SVC(kernel = "linear")
linear_svm.fit(X_train, y_train)
linear_pred = linear_svm.predict(X_test)
linear_acc = accuracy_score(y_test, linear_pred)
print("Accuracy (linear):", linear_acc)

#Poly Model
poly_svm = SVC(kernel = "poly", degree = 3)
poly_svm.fit(X_train, y_train)
poly_pred = poly_svm.predict(X_test)
poly_acc = accuracy_score(y_test, poly_pred)
print("Accuracy (poly):", poly_acc)

#RBF Model
rbf_svm = SVC(kernel = "rbf")
rbf_svm.fit(X_train, y_train)
rbf_pred = rbf_svm.predict(X_test)
rbf_acc = accuracy_score(y_test, rbf_pred)
print("Accuracy (rbf):", rbf_acc)
```

```
Accuracy (linear): 0.9183333333333333
Accuracy (poly): 0.8066666666666666
Accuracy (rbf): 0.9183333333333333
```

In [6]:
```
results = pd.DataFrame({
    "Kernel" : ["Linear", "Polynomial", "RBF"],
    "Accuracy": [linear_acc, poly_acc, rbf_acc]})
print(results.head())
```

```
       Kernel  Accuracy
0      Linear  0.918333
1  Polynomial  0.806667
2         RBF  0.918333
```

In [ ]: